M1: The ITCH 5.0 parser

itchbook walkthrough, milestone 1. How a NASDAQ trading day is turned from a 5.6 GB gzip file into a stream of typed messages, why the parser is built the way it is, and how we know it reads every byte correctly.

What ITCH is

TotalView-ITCH is the feed NASDAQ publishes to tell the world everything that happens on its order book. It is an event log, not a series of snapshots. It never says "the best bid in Apple is $321.92 for 15 shares". It says "order 42 was added: buy 100 Apple at $321.92", then "order 42 had 85 shares executed", then "order 42 was cancelled". Anyone who wants the state of the book has to replay every event from the start of the day, in order, which is exactly what this project does. The feed is anonymous apart from the optional market-participant tag on one message type. It carries every displayed order, which is why rebuilding it gives the full order-by-order (L3) book rather than just totals per price.

A day's file is a sequence of messages, each preceded by a two-byte length. The first byte of every message is its type, a single letter. The 2023 revision of the specification defines 23 types. Every message then carries the same ten-byte header:

- a two-byte stock locate, which is the day's numeric index of the instrument;
- a two-byte tracking number;
- a six-byte timestamp in nanoseconds since midnight Eastern time.

After the header come type-specific fields. All integers are unsigned and big-endian, meaning the most significant byte comes first. Prices are integers with four implied decimal places, so 3219200 means $321.92. Text fields are fixed-width ASCII padded with spaces.

Integer prices matter more than they look. A price stored as a binary floating-point number can't represent most decimals exactly. $0.1 is a repeating fraction in binary, so sums drift and two equal prices can compare unequal. The exchange avoids the problem by sending integers, and the engine never converts them. Every comparison, map key and checksum uses the exchange's own integer.

Reading a message without copying it

The obvious way to parse is to define a struct for each message type, copy the bytes into it field by field, and pass the struct along. For 423 million messages a day, that copies 20 to 50 bytes per message into memory nobody needed, usually to read only two or three of the fields.

itchbook uses *views* instead. A view is a small object holding one pointer, to the first byte of the message inside the input buffer. Each field is a method that reads its bytes at a fixed offset when it is called. `AddOrder::price()` reads four bytes at offset 32. `OrderExecuted::executed_shares()` reads four bytes at offset 19. Constructing a view costs nothing, and a handler that only needs the order reference and the shares never touches the rest of the message. The trade-off is lifetime. A view is only valid while the buffer it points into is unchanged, so handlers must use a message during the callback and must not keep the view. Anything they need later, they copy out as a plain integer. That rule is stated in the header, and the order book respects it: it stores numbers, never views.

Two details of each field read are worth being able to explain. First, alignment. Messages are packed back to back, so a four-byte price can start at any address. The code never casts the byte pointer to an integer pointer, which is undefined behaviour in C++. It copies the bytes into a local integer with `memcpy`, and the compiler turns that into one unaligned load instruction. Second, byte order. The x86 and ARM machines we run on are little-endian and the feed is big-endian, so each load is followed by a byte swap, which is also a single instruction. Whether to swap is decided at compile time from `std::endian`, so on a big-endian machine the swap disappears entirely.

Dispatch: one switch, a strict length check, and templates instead of virtual calls

A message's type byte is examined once, in a `switch` with one case per type. Each case first checks that the frame length equals the length the specification gives for that type, for example 36 bytes for an add order. Only then does it call the handler with the matching view. A wrong length is counted and the message is skipped. It is never decoded, because reading a 35-byte frame as a 36-byte message would read one byte of the next message as a price.

The length check is deliberately strict. A production feed handler might accept longer messages so that new fields can be appended, but for verification we want any disagreement with the spec to be visible. Over two full days the strict check rejected nothing.

The handler is any object with an `on(const AddOrder&)`-style overload for each type it cares about. Deriving from `NullHandler` gives a do-nothing default for the rest. This is static polymorphism. The parser is a template over the handler type, so the compiler sees the concrete handler at every call site, inlines it, and removes the branches for message types the handler ignores. The alternative is a base class with 23 virtual methods. That would cost an indirect call per message and prevent inlining. In a loop that runs 423 million times, that indirection is a measurable fraction of the 7–10 ns the whole parse takes.

Streaming a file larger than memory

The development machine has 8 GB of RAM. One day's file decompresses to 13 GB, so the parser can never hold a day in memory. It works on one fixed buffer of about one megabyte plus 64 KB.

The loop is simple. Read as many bytes as fit. Walk forward through complete frames, handing each to the dispatcher. When the remaining bytes are less than a whole frame, move that partial frame to the start of the buffer, then read again into the space after it. The buffer is sized so this always works. A frame is at most 2 + 65,535 bytes, because the length field is two bytes. The leftover tail is always shorter than one frame, so after compaction there is always at least a megabyte of free space for the next read. Memory use is therefore constant whatever the file size. The copy is at most 64 KB per megabyte read, which is negligible.

Decompression sits underneath as another byte source. The gzip reader uses zlib in streaming mode, feeding it 256 KB of compressed input at a time and asking it to fill the parser's buffer directly. Two edge cases are handled explicitly.

The first is multi-member files. The gzip format allows several compressed streams to be concatenated, and decompressing the file must produce the concatenation of their contents. When zlib reports the end of one stream, the reader resets it and continues with the next.

The second is truncated files. A partial download ends in the middle of a compressed stream. Most code would throw at that point. Here, everything decoded up to the cut is still delivered, the source is flagged as truncated, and the statistics report it. You can develop on a partial file, but you can never mistake one for a full day.

Verification: four checks, two of which caught real bugs

A parser is easy to get almost right. A field read from offset 20 instead of 19 still produces numbers, just wrong ones. The checks below were designed so that this kind of error can't survive.

Field-by-field unit tests. For each of the 23 types, a test encodes a message with a separate writer and checks every accessor. The writer appends fields one after another in spec order, the way a format string would, while the views use explicit offsets, so the layout is described twice, independently. Test values set the top bit of every field width (for example `0xF1E2D3C4` for a four-byte field), so a signed load or a truncated value can't pass.

Coverage of every byte. A reflection function visits each field of a message as a (name, value) pair. One test sets each byte of each message type to a non-zero value, one at a time. It requires that exactly one field changes each time. This proves the fields tile the message with no gap and no overlap. The next check depends on that

guarantee.

Property tests and fuzzing. These were described in the M0 walkthrough. Arbitrary message sequences, cut into chunks of any size, survive the round trip through framing and gzip byte for byte. Arbitrary garbage input never crashes the parser under the sanitizers and is always fully accounted for.

An independent implementation over a whole day. `itch-stats --fingerprint` prints, for every message type, the count and the sum (mod 2^64) of every field across the day. The Python reference parser in `tools/itchref` computes the same table its own way: sequential `struct` format strings, its own framing loop, standard library only. Because every byte belongs to exactly one field, a decoding error anywhere changes at least one sum.

The independent implementation earned its place on its first run. The Python stock-directory layout had one single-character field missing: 38 bytes instead of 39. Every directory message was therefore rejected for the wrong length. The synthetic cross-check caught it immediately and the C++ code was not involved at all, which shows the two implementations really were written separately.

The second bug was in the test of the check. To prove that the cross-check fails when a layout is wrong, a mutation test swaps two adjacent fields in the Python layout and requires the outputs to differ. For the add-with-attribution message this mutation passed silently. Swapping a four-byte integer and a four-byte text field reads exactly the same bytes at exactly the same offsets, so the "mutation" changed nothing. The fix was to detect equal-width swaps and swap the field names instead, which really does break the match. Now every one of the 23 mutations is detected. This is the lesson carried over from holdem-cfr: a check has to be tested by showing that it can fail.

The gate: two full trading days

The M1 gate ran on the Mac over complete NASDAQ days.

For **30 January 2020**:

- `itch-stats` decoded 423,285,709 messages, with zero unknown types, zero wrong lengths and zero truncation.
- The independent Python parser took 702 seconds against 32 seconds for the C++. Its output was byte-for-byte identical to the C++ output: 162 lines, covering 18 counts and 138 per-field sums.
- All 18 per-type counts equal the third-party table published for the same file, and the total equals the published total exactly.

That published source contradicts itself on one count, the number of system-event messages, printing 6 in one place and 5 in another. Both of our implementations count 6. That is also the number of event codes the specification defines for a day (start of messages, start of system hours, market open, market close, end of system hours, end of messages), and all six appear in the file, with timestamps between 03:02:33 and 20:05:00.

For **30 December 2019**: 268,744,780 messages, also clean.

On both days timestamps never decrease from one message to the next.

The run also corrected one belief from the specification research. The file-format specification describes a zero-length frame as an end-of-session marker, but neither file contains one. The day ends with the "end of messages" system event, followed by end of file. The parser accepts both conventions, so nothing needed to change, but the walkthrough says it because an interviewer may know the spec.

Speed, and what it means for the next milestones

On the M2 MacBook Air, the full day took 31.9 seconds end to end: 13.3 million messages per second, 75 ns per message. In-memory microbenchmarks with no decompression put framing and dispatch at about 7.5 ns per message, and decoding every field of every message at about 8.3 ns. Decompression with zlib runs at about 400 MB/s. So roughly 90% of the end-to-end time is zlib, not parsing. Two consequences follow. When M3 compares the order book against CppTrader, both engines have to read the same uncompressed input, or the comparison measures zlib. And if end-to-end throughput matters for the research, the fix is a faster decompressor or decompression on a second core, not a faster parser.

Questions to expect

Why views rather than structs? They decode only the fields that are read and copy nothing. The cost is a lifetime rule, which the type design makes explicit: views are handed to a callback and never stored.

How do you know the offsets are right? Four independent ways. The test writer's sequential layout agrees with the views' explicit offsets. Coverage is complete, with every byte in exactly one field. A separately written Python parser produces an identical per-field checksum over 423 million real messages. The published per-type counts match. And the cross-check has been shown to fail when a layout is wrong.

What happens if the feed contains something unexpected? A wrong length or unknown type is counted and skipped, never decoded. A truncated file is flagged. Nothing is silently dropped: every byte of input is accounted for as a delivered message, a rejected frame, or a reported incomplete tail, and the fuzzer checks that

invariant on arbitrary input.

Why is the Python reference so slow, and does that matter? It is a plain interpreted loop, about 1.7 µs per message. It doesn't need to be fast: it runs once per milestone gate, and simplicity is what makes it a credible second opinion.